

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему
**ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ КАРТОЧНУЮ
ИГРУ «БЛЕКДЖЕК»**

БГУИР КП 6-05 06 12 01 020 ПЗ

Студент

А. Б. Кляус

Руководитель

С. В. Болтак

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Сравнительный анализ существующих решений.....	6
1.2 Постановка задачи	8
2 Проектирование программы.....	10
2.1 Проектирование структуры программы.....	10
2.2 Проектирование системы сетевого взаимодействия.....	11
3 Программная реализация	13
3.1 Описание разработанных модулей	13
3.2 Описание сетевого модуля	15
4 Тестирование.....	17
5 Руководство пользователя	19
Заключение.....	30
Список использованных источников.....	31
Приложение А (обязательное) Листинг кода с комментариями.....	32
Приложение Б (обязательное) Схема алгоритма работы системы	55

ВВЕДЕНИЕ

В эпоху цифровизации традиционные настольные игры активно переходят в виртуальное пространство. Одним из востребованных представителей этого класса является блекджек – классическая карточная игра, сочетающая математические закономерности с высокодинамичным процессом. Разработка современного веб-приложения для блекджека представляет собой комплексную инженерную задачу на стыке проектирования распределенных систем, реализации сетевых протоколов реального времени и построения масштабируемых интерфейсов. Актуальность работы обусловлена высоким спросом на интерактивные платформы, способные обеспечивать мгновенный двусторонний обмен данными без задержек, свойственных классической http-модели.

Объектом исследования выступают технологии построения многопользовательских веб-систем реального времени на базе асинхронного подхода. Предметом исследования является процесс проектирования, программной реализации и тестирования игрового сервиса с использованием технологического стека, включающего node.js [1], express [2][3], typescript [4], socket.io [5][6], postgresql [7] и redis [8].

Цель проекта заключается в создании надежной программной системы, предоставляющей пользователям возможности безопасной авторизации, создания виртуальных комнат с гибкой настройкой правил, ведения игровых сессий в реальном времени и сохранения учетных данных в энергонезависимом хранилище.

Практическая значимость разработки определяется возможностью применения реализованных решений в качестве основы для построения более широкого класса многопользовательских игровых платформ. Выработанные подходы к организации серверной логики, управлению состоянием игровых сессий и обеспечению отказоустойчивости могут быть перенесены на смежные предметные области, требующие низкой задержки передачи данных и высокой степени согласованности распределенного состояния.

Для достижения поставленной цели необходимо решить ряд последовательных задач, начиная с проведения сравнительного анализа программных аналогов и формирования комплекса требований к веб-приложению. Далее требуется разработать модульную архитектуру системы, спроектировать протокол полнодуплексного взаимодействия на базе websocket [9] и продумать схему базы данных совместно с механизмами кэширования. Практическая реализация комплекса должна сопровождаться строгой типизацией данных, после чего необходимо провести полномасштабное тестирование готового продукта и разработать техническую документацию по его эксплуатации и развертыванию.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Приступая к проектированию архитектуры многопользовательской игровой веб-платформы, необходимо провести анализ существующих на рынке программных продуктов, предоставляющих функционал карточных игр в реальном времени. Рассмотрим основные аналоги разрабатываемой системы, представленные в различных сегментах индустрии.

1. Программные клиенты крупных коммерческих операторов, таких как *pokerstars* или *888casino*, представляют собой высоконагруженные системы, ориентированные на непрерывную многопользовательскую сессию. Данные решения обеспечивают поддержку миллионов одновременных подключений, предлагают широкий выбор игровых комнат и отличаются минимальной сетевой задержкой за счет использования бинарных протоколов связи. Основным недостатком является необходимость скачивания и установки специализированного программного обеспечения, что исключает возможность мгновенной игры непосредственно в браузере и существенно повышает порог входа для новой аудитории.

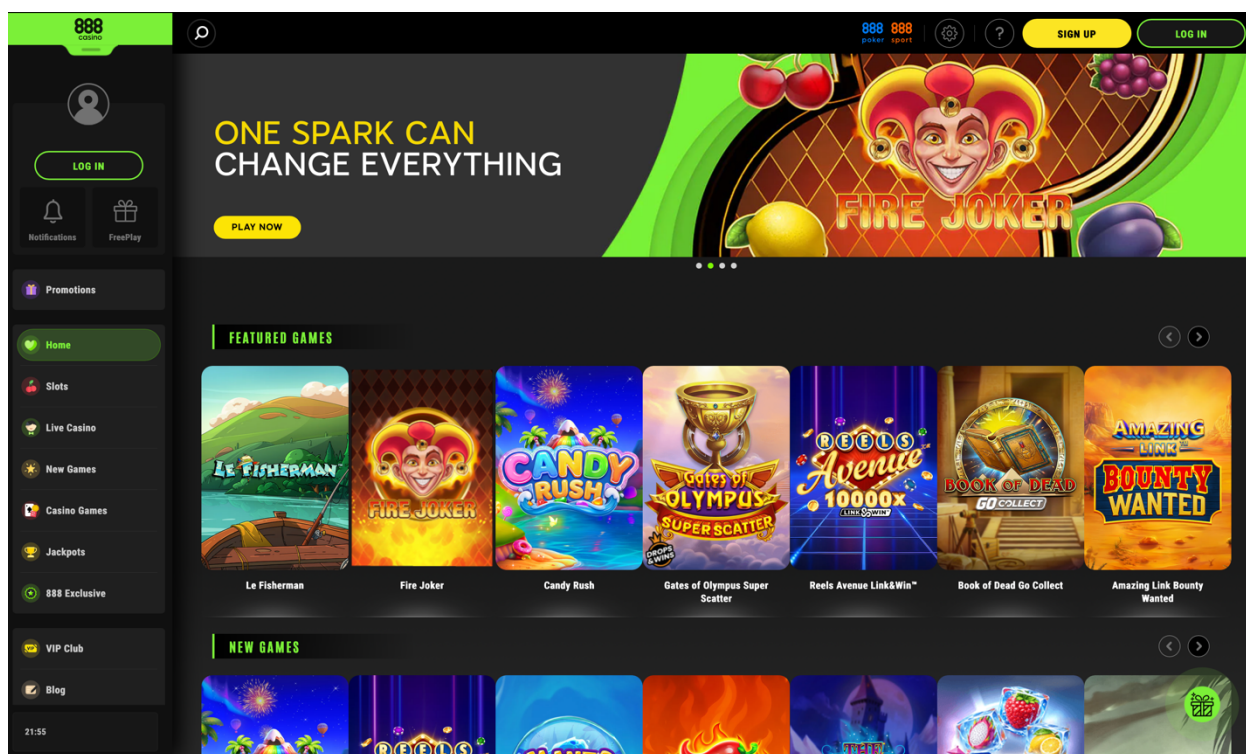


Рисунок 1.1 – 888casino

2. Платформы для интеграции браузерных онлайн-казино, разрабатываемые провайдерами уровня *evolution gaming* или *playtech*, специализируются на предоставлении доступа к карточным играм через веб-интерфейс. Такие системы используют сложные графические webgl-

интерфейсы или потоковое видео с живыми дилерами. Они характеризуются высокой степенью защиты данных и строгим контролем игрового процесса. Однако подобные решения обладают избыточными требованиями к аппаратному обеспечению и пропускной способности сети пользователя, а также представляют собой тяжеловесные коммерческие модули, не предполагающие создания изолированных частных комнат самими игроками.

3. Браузерные социальные казино и многопользовательские веб-симуляторы, например приложения формата *blackjackist* или *zynga poker*, ориентированы на массовую аудиторию и казуальный игровой процесс. Эти продукты легко запускаются в любом современном браузере без предварительной установки и обладают интуитивно понятной системой взаимодействия. К существенным недостаткам таких решений относятся слабая консистентность игрового состояния при нестабильном интернет-соединении, перегруженность интерфейса сторонними социальными функциями и частые отступления от классической математической модели игры в угоду удержанию внимания пользователей.

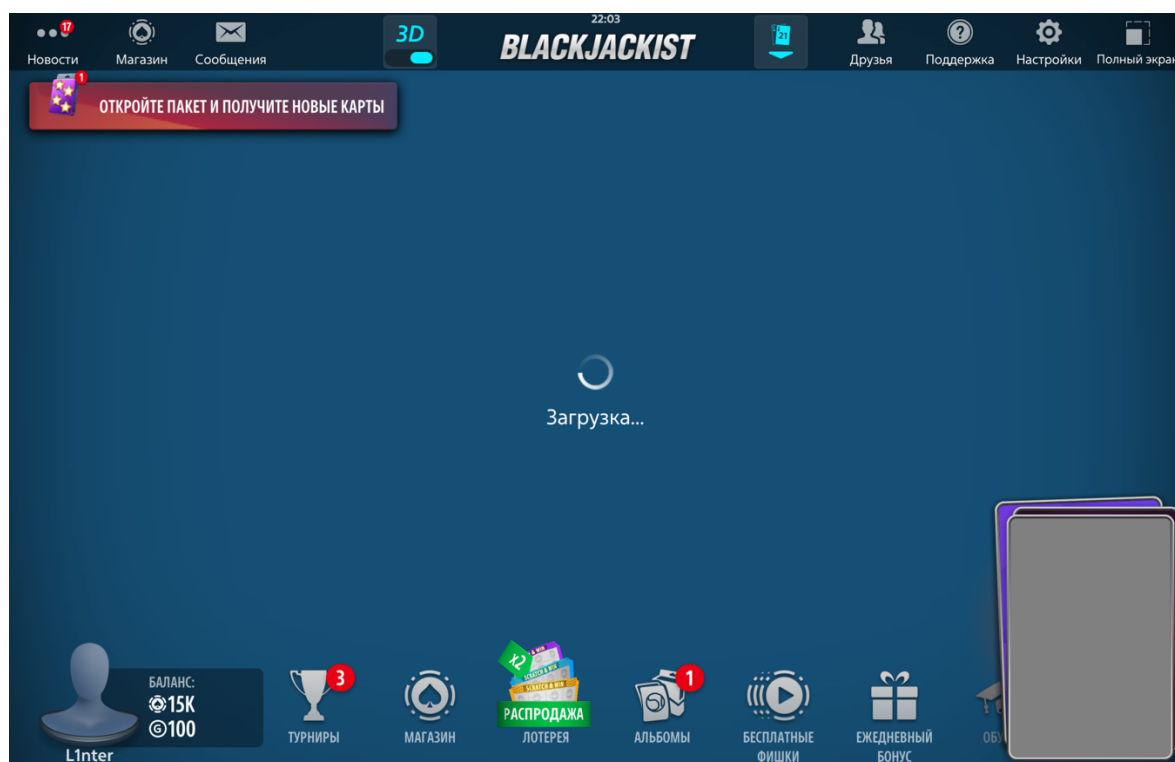


Рисунок 1.2 – *BlackJackist*

Таким образом, анализ существующих программных решений показывает, что на рынке преобладают либо тяжеловесные нативные клиенты, либо избыточно перегруженные коммерческие симуляторы. Разрабатываемое веб-приложение призвано занять нишу легковесных платформ, объединяющих строгую реализацию классических правил блекджека, возможность создания пользовательских комнат, низкую сетевую задержку и

абсолютную кроссплатформенность без необходимости установки стороннего программного обеспечения.

1.2 Постановка задачи

На основании аналитического обзора предметной области сформулировано техническое задание на разработку многопользовательского игрового веб-приложения blackjack. Система представляет собой клиент-серверный веб-комплекс с интерфейсом, доступным через современные интернет-браузеры без необходимости установки дополнительного программного обеспечения на стороне пользователя.

Функциональные требования к программному комплексу распределены по четырем основным логическим модулям. Модуль управления пользователями обеспечивает регистрацию новых учётных записей с обязательной валидацией уникальности псевдонима и адреса электронной почты, безопасное хранение паролей в виде хэш-значений [10], а также авторизацию пользователей. Для защиты от межсайтового скриптинга выдаваемые сессионные токены должны сохраняться исключительно в заголовках http-only cookie. Дополнительно данный модуль отвечает за обработку процедуры выхода из учетной записи с обязательным аннулированием активной сессии на стороне сервера.

Модуль управления игровыми комнатами позволяет авторизованным пользователям просматривать список активных столов и создавать новые комнаты с заданными параметрами, среди которых уникальное название, текстовое описание, ограничение максимального числа участников, количество колод в шузе, а также признак приватности и соответствующий пароль доступа. Кроме того, данный компонент организует вход пользователей в существующие комнаты, проверку доступов и корректную процедуру выхода из лобби с уведомлением других участников.

Модуль игрового движка отвечает за строгое соблюдение классических правил игры в блекджек и последовательное переключение игровых фаз, включая прием ставок, раздачу начальных карт, поочередное выполнение ходов пользователями, автоматический ход дилера до достижения семнадцати очков или выше, а также подведение итогов раунда с расчетом выплат. Движок реализует полную поддержку таких игровых действий, как взятие карты, фиксация текущего счета, удвоение ставки с получением одной карты, разделение пары одинаковых карт и отказ от игры с возвратом половины сделанной ставки. При расчете суммарного количества очков в руке алгоритм должен учитывать динамическое изменение ценности туза в зависимости от текущей вероятности перебора. Помимо перечисленных игровых действий, движок обязан корректно обрабатывать граничные ситуации: мгновенную победу при получении блекджека с первых двух карт, ничью при равенстве очков у игрока и дилера, а также автоматическое завершение хода в случае достижения счета в двадцать одно очко.

Модуль коммуникации дополняет игровой процесс и обеспечивает функционирование изолированного текстового чата внутри каждой игровой комнаты, позволяя участникам обмениваться сообщениями в режиме реального времени без сохранения истории переписки после закрытия лобби.

К разрабатываемому программному комплексу предъявляется ряд жестких нефункциональных и технических требований. В области производительности и латентности устанавливается предел времени на обработку сервером любого сетевого события и последующую трансляцию обновленного состояния всем клиентам в комнате, который не должен превышать 50 миллисекунд без учета задержек физических каналов связи. Столь жесткое ограничение обусловлено тем, что заметные задержки в передаче игровых событий нарушают синхронность восприятия игрового процесса участниками и негативно сказываются на общем качестве пользовательского опыта.

Консистентность данных достигается за счет транзакционного выполнения любого изменения состояния на стороне сервера с последующей централизованной рассылкой обновлений. Хранение и оперативное обновление промежуточных игровых состояний, текущих параметров комнат и активных сессий реализуется в оперативной базе данных redis. Долгосрочное хранение профилей пользователей, их учетных данных и основного финансового баланса организуется в реляционной базе данных postgresql. Подобное разделение между быстрым энергозависимым хранилищем и надежной реляционной базой данных позволяет одновременно обеспечить высокую скорость доступа к оперативным данным и гарантированную сохранность критически важной пользовательской информации.

Надёжность и отказоустойчивость системы гарантируют, что при кратковременном обрыве соединения длительностью до 10 секунд клиентский скрипт осуществит автоматическое переподключение к серверу, запросит актуальный снимок состояния игровой комнаты и полностью восстановит пользовательский интерфейс без потери данных. Механизм восстановления сессии реализуется путем хранения идентификатора комнаты и токена пользователя в защищенном локальном хранилище браузера, что позволяет клиенту однозначно идентифицировать себя при повторном подключении и получить от сервера полный снимок текущего состояния игры.

Требования к безопасности предписывают выполнение всей ключевой игровой логики, включая подсчет очков, генерацию и перемешивание колоды, принятие решений за дилера и валидацию ходов, исключительно на стороне сервера. Клиентская часть приложения выступает только в роли интерфейса визуализации и не имеет технической возможности напрямую влиять на внутренние переменные и состояние игрового движка. Такой подход исключает целый класс уязвимостей, связанных с подменой клиентских данных: любая попытка передать на сервер недопустимое игровое действие или некорректное значение ставки будет отклонена на этапе серверной валидации до применения каких-либо изменений к состоянию комнаты.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1 Проектирование структуры программы

Архитектура разрабатываемой программной системы построена на принципах модульного событийно-ориентированного проектирования и структурно разделена на два фундаментальных контура: серверный компонент в среде выполнения *node.js* и реактивный клиентский компонент, исполняемый в браузере. Подобное физическое и логическое разделение позволяет минимизировать сетевые издержки, обеспечить высокую масштабируемость и четко разграничить зоны ответственности между интерфейсом визуализации и защищенным ядром бизнес-логики.

Внутренняя структура серверного приложения организована в соответствии со слоистой архитектурной парадигмой. Первый базовый уровень представляет собой слой маршрутизации и обработки входящих *http*-запросов, реализованный на базе высокопроизводительного фреймворка *express* [2][3]. Он отвечает за первичную инициализацию веб-сервера, конфигурацию промежуточного программного обеспечения для безопасного разбора сессионных *cookie* и *json*-пакетов. Кроме того, этот уровень обслуживает маршруты авторизации и осуществляет динамический рендеринг веб-страниц с использованием шаблонизатора *handlebars* [11].

Второй уровень представлен слоем высокоскоростного двунаправленного взаимодействия, функционирующим под управлением библиотеки *socket.io* поверх стандарта *websocket*[9]. Данный компонент централизованно управляет жизненным циклом постоянных сетевых соединений. На этапе авторизационного рукопожатия система извлекает сессионные токены, верифицирует их и привязывает сокеты к профилям игроков. Далее этот слой распределяет активные подключения по изолированным виртуальным комнатам, организуя мгновенную широковеб-трансляцию асинхронных игровых событий всем участникам конкретного стола.

Центральное место в архитектуре занимает слой изолированной бизнес-логики, вычислительным ядром которого служит специализированный математический движок карточной игры. Этот модуль инкапсулирует в себе защищенные алгоритмы генерации и тасования виртуальных колод, математические функции вычисления достоинства комбинаций с учетом переменной ценности тузов, а также строгий конечный автомат состояний комнаты. Конечный автомат гарантирует безопасную и детерминированную смену игровых фаз: прием начальных ставок, раздачу карт, обработку пользовательских ходов, автоматические действия дилера и финальный расчет выплат.

За надежное сохранение информации и бесперебойное управление контекстом отвечает слой доступа к данным, функционально разделенный на два независимых репозитория. Долгосрочное персистентное хранение учетных записей, хэшированных паролей и актуальных финансовых балансов

пользователей делегировано реляционной базе данных *postgresql* [7]. Параллельно с этим хранение высокодинамичного состояния активных столов и промежуточных игровых решений перенесено в *in-memory* хранилище *redis* [8], что позволяет полностью исключить ресурсоемкие дисковые операции в процессе активной игры.

Клиентская часть программного комплекса спроектирована как автономное реактивное приложение, управляемое входящим потоком сетевых данных. Ее внутренние модули отвечают за поддержание стабильного соединения, автоматическое переподключение и обработку серверных событий. Специализированные контроллеры осуществляют плавную мутацию *dom*-дерева веб-страницы для визуализации раздачи карт, обновления балансов и вывода итогов раунда. Синхронно с этим клиентские обработчики перехватывают управляющие действия пользователя, трансформируя выбор фишек, игровые ходы и сообщения чата в строго типизированные запросы.

Глобальное взаимодействие всех компонентов организуется посредством параллельного использования двух независимых каналов коммуникации. Первичный *http*-протокол применяется исключительно для загрузки базового интерфейса и безопасного проведения процедуры авторизации. Вторичный канал, опирающийся на постоянные сетевые сокеты, монопольно обслуживает непрерывный игровой процесс. В этой структуре серверная часть выступает в роли единого координатора, который верифицирует поступающие команды, фиксирует изменения в оперативной памяти и производит итоговую транзакционную запись результатов в дисковое хранилище, гарантируя абсолютную консистентность всей системы.

2.2 Проектирование системы сетевого взаимодействия

Сетевое взаимодействие является наиболее критичным узлом системы реального времени. Традиционная модель *http*, основанная на парадигме одиночных запросов и ответов, непригодна для фазы активного игрового процесса, поскольку сервер не обладает механизмом самостоятельной инициации отправки данных клиенту в момент, когда другой участник берет карту или дилер завершает раунд. Для решения этой проблемы спроектирована событийно-ориентированная архитектура на базе протокола *websocket* [9] с использованием библиотеки *socket.io* [5].

Взаимодействие начинается с этапа установления связи, также известного как *handshake*. При переходе пользователя на страницу игровой комнаты клиентская библиотека *socket.io* отправляет первичный *http*-запрос, содержащий заголовок *upgrade: websocket*. Сервер извлекает из него сессионный токен, выполняет его валидацию через связку хранилищ *redis* и *postgresql* и, в случае успеха, подтверждает переключение протокола. Установленной сессии сокета присваивается уникальный идентификатор, записываемый в свойство *socket.id*, а верифицированные данные пользователя надежно привязываются к объекту *socket.data* для быстрого доступа при последующих операциях.

Исходящий поток данных от клиента к серверу строго регламентирован и обрабатывается через фиксированный набор событий (*client-to-server events*). Процесс подключения инициируется событием *room_join*, в рамках которого передается параметр *roomid*, а для принудительной синхронизации интерфейса предусмотрен запрос *get_room_state*, возвращающий актуальный снимок стола. Непосредственное управление игровым процессом осуществляется посредством трансляции команд: *place_bet* для фиксации ставки в соответствующей фазе, *hit* для запроса дополнительной карты и *stand* для завершения набора карт текущей рукой. Кроме того, реализована поддержка дополнительных стратегических действий, таких как событие *double* для удвоения ставки с автоматическим получением ровно одной карты, *split* для разделения парных карт на две независимые руки и *surrender* для добровольной сдачи с возвратом пятидесяти процентов от суммы ставки. Отправка текстовых сообщений во внутриигровой чат обслуживается отдельным событием *chat_message*.

Входящий поток данных от сервера к клиенту (*server-to-client events*) инкапсулирует команды асинхронного управления интерфейсом. Сервер транслирует событие *room_state*, содержащее полный *json*-снимок стола при первичном подключении, после чего рассылает сигнал *game_started*, информирующий о начале нового раунда и старте раздачи карт. В процессе активной игры клиенты получают атомарные обновления через событие *player_action*, фиксирующее изменение состояния руки после хода оппонента, а также *turn_changed* для уведомления о переходе права действия к следующему участнику. Поэтапная трансляция процесса набора карт дилером осуществляется через событие *dealer_play*. По завершении игрового цикла сервер формирует детализированный отчет *round_result* для всех участников и инициирует событие *betting_phase* для перевода комнаты обратно в стадию сбора ставок. Вспомогательный функционал протокола обеспечивает ретрансляцию текстовых данных через событие *chat_message*, рассылку сервисных уведомлений *player_joined* и *player_left* о перемещениях участников, а также отправку события *error*, содержащего код и описание ошибки при попытках совершения недопустимых действий.

Спроектированный протокол сводит к минимуму объем передаваемых данных, транслируя преимущественно изолированные дельта-обновления состояния вместо пересылки всего контекста комнаты. Это гарантирует высокую отзывчивость пользовательского интерфейса и бесперебойность игрового процесса даже при значительных колебаниях качества интернет-соединения.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Файловая структура проекта представлена на рисунке 3.1.

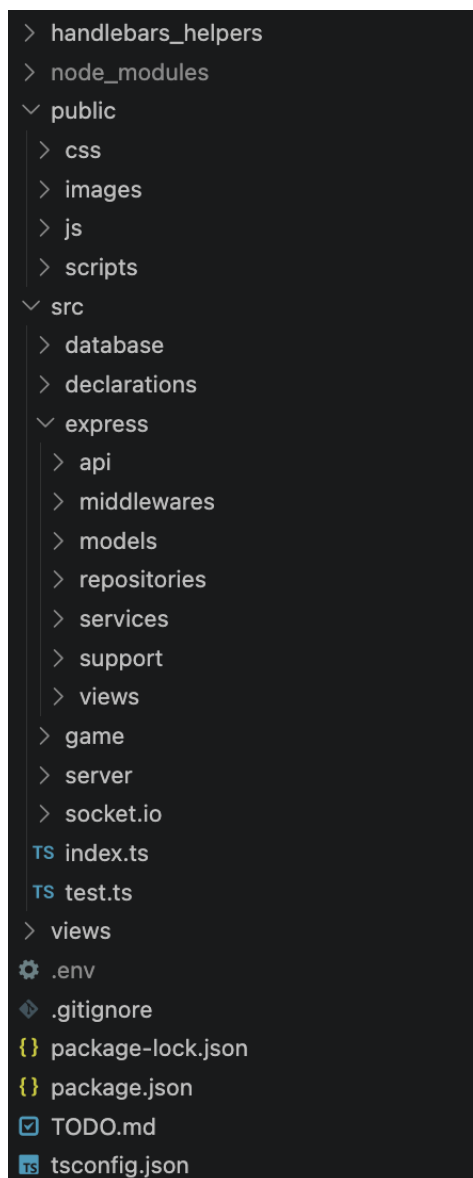


Рисунок 3.1 – Файловая структура проекта

3.1 Описание разработанных модулей

Разработанное веб-приложение построено на основе модульной архитектуры, обеспечивающей строгое разделение ответственности между компонентами клиентской и серверной частей. Программное средство состоит из нескольких взаимосвязанных логических блоков, каждый из которых инкапсулирует определенный набор функций. Подобная организация исходного кода упрощает отладку, профилирование и последующее масштабирование системы, позволяя модифицировать отдельные подсистемы без риска нарушения целостности всего комплекса.

Модуль доступа к данным отвечает за операции взаимодействия с базами данных и предоставляет унифицированную абстракцию над различными механизмами хранения информации. Серверная часть комплекса использует гибридную инфраструктуру. Долгосрочное и надежное хранение реализовано через специализированный репозиторий на базе реляционной субд *postgresql*, который управляет учетными записями, хэшированными паролями и актуальными финансовыми балансами игроков. Оперативное хранение возложено на репозитории резидентной системы *redis*. Модуль управления сессиями отвечает за кэширование авторизационных токенов, формируемых криптографической утилитой генерации, а компонент управления комнатами контролирует метаданные активных столов и списки участников, обеспечивая сверхбыстрый доступ к изменяемому контексту без задержек дискового ввода-вывода.

Модуль бизнес-логики инкапсулирует ключевую функциональность приложения и содержит алгоритмы, реализующие строгие математические правила карточной игры. Центральным элементом этого слоя выступает программный движок блекджека, полностью абстрагированный от сетевых протоколов. В его задачи входит генерация стандартизированного карточного шуза заданного объема, криптографически надежное перемешивание карт на основе алгоритма фишера-йейтса [12] и функция оценки текущего состояния рук. Алгоритм подсчета очков производит динамическое вычисление номиналов, автоматически понижая ценность тузов с одиннадцати до одного очка при угрозе перебора. Движок непрерывно управляет конечным автоматом состояний виртуального стола, регламентируя переходы между фазами приема ставок и активной игры, а также выполняет итоговый расчет выигрышей с применением классических коэффициентов выплат и синхронизирует результаты с модулем хранения.

Модуль сетевого взаимодействия и управления лобби координирует распределение участников и гарантирует мгновенную доставку игровых событий. Он объединяет клиентские скрипты управления интерфейсом лобби и серверные контроллеры, построенные поверх библиотеки *socket.io*. Данный компонент позволяет пользователям запрашивать актуальный список доступных столов, создавать новые приватные или публичные комнаты с индивидуальными лимитами числа участников и количества колод, а также безопасно подключаться к открытым партиям посредством верификации паролей. Для обеспечения безопасности этот же модуль реализует механизм защиты от множественных подключений: при входе пользователя в систему с нового устройства инициируется принудительное отключение старых сессий с автоматическим удалением устаревших сокетов из активных игровых комнат.

Модуль маршрутизации и пользовательского интерфейса обеспечивает визуальное представление информации и первичную обработку *http*-запросов. На стороне сервера маршрутизация выстроена с использованием веб-фреймворка *express* [3], обслуживающего *rest api* конечные точки авторизации, регистрации и управления профилем. Динамическое

формирование страниц реализовано посредством шаблонизатора *handlebars*. Специализированный программный обработчик контекста анализирует входящие запросы на предмет наличия валидного сессионного токена в файлах *cookie* и выполняет условный рендеринг. При успешной проверке пользователю отображаются защищенные элементы навигации и открывается доступ к игровому лобби, тогда как неавторизованным клиентам предоставляются исключительно формы базового доступа. На стороне браузера реактивные скрипты перехватывают ответы сервера и осуществляют динамическую перестройку структуры объектной модели документа без необходимости полной перезагрузки веб-страницы.

3.2 Описание сетевого модуля

Сетевой модуль выступает центральным коммуникационным ядром разрабатываемого программного комплекса, обеспечивая полнодуплексный обмен данными в реальном времени. Его логика физически разделена на два компонента: серверные контроллеры, реализованные в файле *socket.io.ts*, и клиентский реактивный скрипт *game.js*. Процесс взаимодействия начинается с инициализации соединения на стороне браузера, где вызов функции подключения осуществляется с явным указанием параметра *transports: ["websocket"]*. Это принудительно устанавливает стандарт *websocket* в качестве единственного транспортного механизма, что позволяет полностью исключить избыточные резервные *http*-опросы и минимизировать сетевые задержки при передаче игровых пакетов.

Архитектура клиентского приложения выстроена на реактивной обработке входящих событий от сервера. При первичном подключении или принудительном запросе синхронизации клиент получает событие *room_state*, содержащее сериализованный снимок текущего стола. Обработчик этого события кэширует информацию в локальную переменную и запускает метод комплексного рендеринга. Данный алгоритм анализирует текущую фазу игры и при необходимости скрывает вторую карту дилера, заменяя её псевдокодом, после чего последовательно перестраивает структуру документа. Происходит отрисовка карт дилера и всех участников, обновление счетчика оставшихся в колоде карт и синхронизация финансовых балансов пользователей в боковой панели интерфейса.

Динамика активного игрового процесса обеспечивается обработкой атомарных событий. Сигнал *player_action* транслируется сервером после каждого успешного хода любого из участников. Пакет данных содержит идентификатор игрока, обновленный массив его раздач и индекс активной руки, что позволяет клиентскому скрипту выполнять точечный рендеринг конкретного слота без полной перерисовки стола. Активная рука визуально выделяется стилистическим классом, а текстовые блоки отображают текущую сумму очков и статус раздачи. Передача права хода обслуживается событием *turn_changed*, которое обновляет глобальный идентификатор активного участника и вызывает функцию переоценки доступности элементов

управления. Метод строго проверяет контекст стола: кнопки специфических действий, таких как удвоение ставки или разделение парных карт, активируются исключительно при соблюдении правил блекджека (наличие ровно двух карт, совпадение рангов при сплите) и совпадении идентификатора ходящего с локальным пользователем.

Завершение раунда координируется через обработку события `round_result`. При его получении клиентская часть блокирует интерфейс управления ходами, полностью раскрывает закрытые карты дилера и формирует детализированную сводку в модальном окне. Алгоритм дифференцирует исходы раунда (натуральный блекджек, победа, ничья или поражение), подставляя соответствующие визуальные индикаторы, и применяет итоговые изменения к балансам игроков на столе. Окно результатов демонстрируется пользователям в течение шести секунд, после чего автоматически скрывается системным таймером, подготавливая интерфейс к новой фазе сбора ставок. Параллельно с визуальным отображением итогов клиентский скрипт направляет на сервер подтверждение получения результатов, что позволяет серверной части точно отслеживать готовность всех участников и своевременно инициировать переход к следующему раунду. В случае если баланс игрока по итогам раунда опускается ниже минимально допустимого порога ставки, интерфейс дополнительно отображает соответствующее уведомление и блокирует возможность участия в следующем раунде до пополнения счета.

Для корректного визуального представления раздач в модуле реализована вспомогательная утилита форматирования. Функция перехватывает строковые серверные идентификаторы карт (например, «ah» или «td») и преобразует их в html-разметку с использованием соответствующих юникод-символов мастей. При этом для червовых и бубновых карт алгоритм динамически присваивает инлайн-класс `card-red`, окрашивающий текст в красный цвет, что обеспечивает интуитивно понятное и аутентичное восприятие виртуальной колоды. Подобный подход к кодированию карт позволяет полностью абстрагировать визуальный слой от внутреннего представления данных: серверу достаточно передавать компактные двухсимвольные идентификаторы, тогда как вся логика преобразования в читаемую разметку сосредоточена на клиентской стороне. Это существенно сокращает объем передаваемых по сети данных и упрощает поддержку кодовой базы, поскольку любые изменения в визуальном оформлении карт не затрагивают серверную логику. Помимо форматирования мастей, утилита также обрабатывает специальные случаи отображения: закрытые карты дилера рендерятся с использованием отдельного `placeholder`-элемента, визуально обозначающего рубашку, что сохраняет целостность игровой механики сокрытия информации на уровне интерфейса.

4 ТЕСТИРОВАНИЕ

Тестирование разработанного многопользовательского веб-приложения проводилось с целью комплексного подтверждения корректности функционирования серверной бизнес-логики, строгого соблюдения математических правил карточной дисциплины, проверки стабильности двунаправленного сетевого взаимодействия и оценки устойчивости платформы к попыткам несанкционированной манипуляции данными со стороны клиентского интерфейса.

В рамках проверки модуля управления пользователями оценивалась корректность обработки запросов на регистрацию и авторизацию. При передаче валидных учетных данных сервер успешно осуществляет криптографическое хэширование пароля, создает новую запись в реляционной базе данных postgresql и возвращает http-статус успешной обработки вместе с уникальным сессионным токеном, который помещается в защищенные файлы cookie. В случае попытки регистрации с использованием уже существующего в системе псевдонима или адреса электронной почты контроллер корректно прерывает транзакцию, возвращая статус ошибки с соответствующим информационным сообщением, что исключает дублирование профилей.

Тестирование подсистемы игровых комнат подтвердило правильность алгоритмов распределения игроков. При создании приватного стола с заданными параметрами, такими как пользовательское название, пароль, количество колод и ограничение на число участников, система безошибочно формирует сериализованные структуры в кэш-памяти redis и возвращает создателю уникальный строковый идентификатор комнаты. Попытки несанкционированного входа в закрытую комнату с передачей некорректного пароля через api успешно блокируются на этапе проверки метаданных стола, после чего клиенту отказывается в доступе без инициализации сетевого сокета.

Основополагающим этапом стала верификация изолированного игрового движка, которая включала как оценку пользовательских сценариев, так и модульное тестирование чистых математических функций на базе встроенных средств среды node.js. Особое внимание уделялось алгоритму динамического подсчета очков. Система безошибочно обрабатывает ситуации с «мягкой» рукой: например, при стартовой раздаче туза и пятерки суммарный счет корректно фиксируется как шестнадцать очков. При последующем запросе дополнительной карты, приводящем к потенциальному перебору, алгоритм автоматически трансформирует ценность туза из одиннадцати очков в одно, сохраняя статус руки активным. В ситуациях, когда добор карты приводит к неизбежному превышению лимита в двадцать одно очко, конечный автомат мгновенно переводит руку в статус перебора и принудительно передает право хода следующему участнику.

Проверка дополнительных игровых стратегий показала полное соответствие классическим правилам. При активации удвоения ставки сервер валидирует наличие достаточного баланса в оперативной сессии игрока,

корректно списывает требуемую сумму, выдает строго одну дополнительную карту и завершает набор для текущей руки. Логика контроля очередности ходов исключает возможность возникновения состояния гонки: любые попытки клиента отправить команду на взятие карты или остановку вне своей очереди немедленно отклоняются сервером с отправкой ответного события об ошибке, так как идентификатор ходящего жестко зафиксирован в состоянии виртуальной комнаты. Проверка автоматического алгоритма дилера подтвердила, что виртуальный крупье последовательно добывает карты из колоды вплоть до достижения суммы в семнадцать или более очков, после чего гарантированно останавливается для финального сравнения результатов.

Помимо функциональных проверок, проведено тестирование механизмов обработки внештатных ситуаций, в частности — устойчивости системы к внезапным обрывам сетевого соединения. При симуляции потери сети клиентом серверный модуль корректно перехватывал событие отключения. Система автоматически завершала ход отключившегося пользователя, передавая право действия следующему участнику, что полностью исключало блокировку игрового процесса. Дополнительно тесты подтвердили консистентность данных: итоговые изменения балансов, рассчитанные в оперативном кэше Redis, безошибочно синхронизировались с реляционным хранилищем PostgreSQL, гарантируя сохранность финансовых показателей при любых сценариях прерывания сессии.

Завершающим этапом стала проверка безопасности программного комплекса. Целенаправленные попытки модификации локальных переменных клиентского интерфейса, таких как текущий финансовый баланс или право хода, с использованием инструментов разработчика в браузере приводили исключительно к визуальным искажениям на устройстве злоумышленника. При попытке совершить фактическое игровое действие с подмененными параметрами серверная часть немедленно пресекала транзакцию, опираясь исключительно на криптографически защищенный токен сессии и доверенное состояние стола в оперативной памяти сервера. Дополнительно была проверена устойчивость системы к повторному воспроизведению перехваченных сетевых пакетов: попытка переотправки ранее записанного события игрового действия отклонялась сервером ввиду несоответствия текущей фазы игры и версии состояния комнаты, зафиксированной в момент первоначальной отправки. Совокупность проведенных испытаний подтверждает, что разработанная архитектура с централизованным хранением доверенного состояния на стороне сервера и строгой валидацией каждого входящего события обеспечивает высокую степень защищенности игровой платформы от широкого спектра клиентских атак и непреднамеренных ошибок пользовательского окружения.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Настоящее руководство предназначено для ознакомления пользователей с графическим интерфейсом, логикой навигации и механизмами обратной связи многопользовательского игрового веб-приложения. Взаимодействие с программным комплексом начинается с запуска браузера и перехода на главную страницу системы.

При первичном посещении ресурса пользователю отображается базовое состояние главного меню. Интерфейс навигационной панели динамически адаптируется под текущий статус сессии. Для посетителей, не прошедших процедуру идентификации, главное меню содержит исключительно навигационные ссылки для перехода к формам регистрации и авторизации, в то время как доступ к игровому функционалу лобби остается программно скрытым.

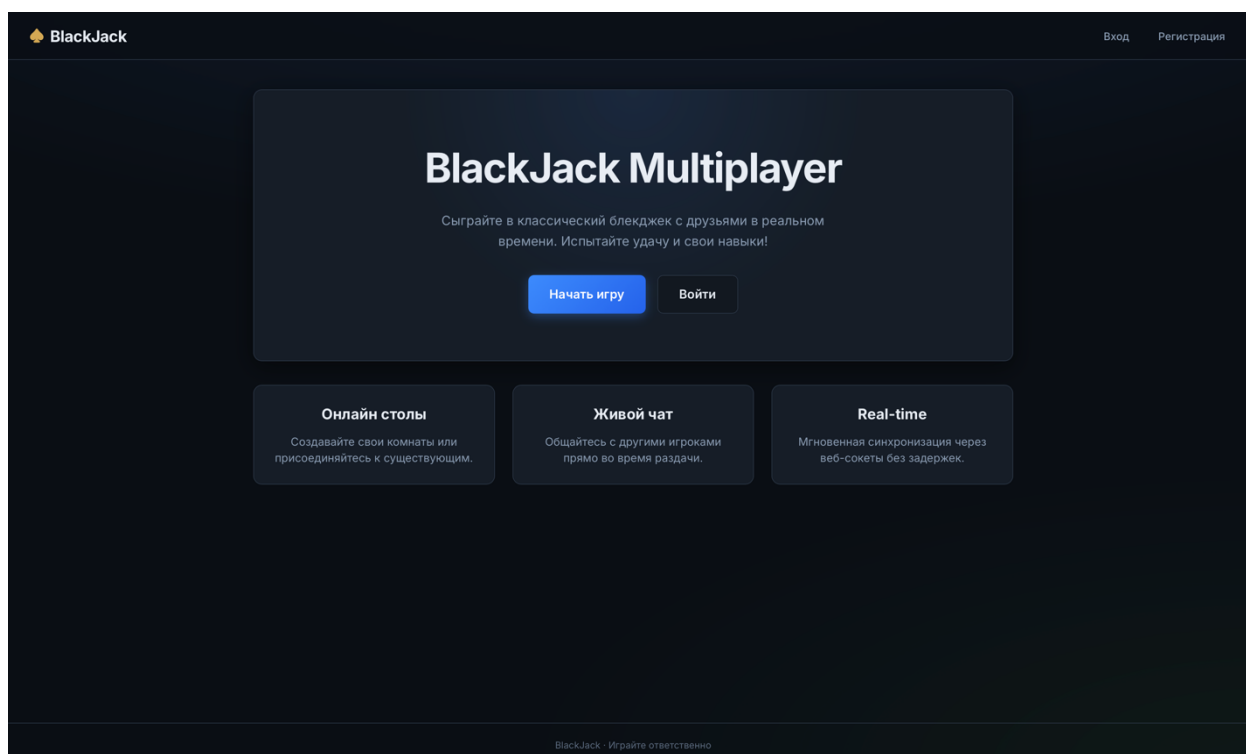


Рисунок 5.1 – Интерфейс главного меню для неавторизованного пользователя

```
GET / HTTP/1.1
HTTP/1.1 302 Found (text/html)
GET /main HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.2 – Запрос на получение основной страницы

Для создания новой учетной записи необходимо заполнить форму регистрации, указав уникальный псевдоним, адрес электронной почты и дважды введя пароль. При заполнении формы система обеспечивает моментальную обратную связь. Если пользователь вводит несовпадающие пароли, локальный скрипт немедленно выводит текстовое предупреждение под полями ввода, предотвращая отправку данных.

```
GET /authorization/register HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.3 – Запрос на получение меню регистрации

Рисунок 5.4 – Индикация ошибки несовпадения паролей на форме регистрации

В случае возникновения исключений на стороне сервера, таких как попытка регистрации уже существующего имени пользователя, система выводит соответствующее сообщение об ошибке внутри самой формы, сохраняя корректно заполненные поля для быстрого исправления.

Рисунок 5.3 – Индикация серверной ошибки на форме регистрации

Доступ к существующему профилю осуществляется через форму авторизации. При вводе неверного пароля или несуществующего логина система аналогичным образом отображает локальное предупреждение об ошибке валидации, не требуя полной перезагрузки страницы.

```
GET /authorization/login HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.4 – Запрос на получение меню входа

Рисунок 5.5 – Индикация ошибки валидации на форме авторизации

```
POST /api/authorization/login HTTP/1.1 , JSON (application/json)
HTTP/1.1 401 Unauthorized , JSON (application/json)
```

Рисунок 5.6 – Попытка зайти с невалидными данными

```
Member: identifier
  [Path with value: /identifier:ivan]
  [Member with value: identifier:ivan]
  String value: ivan
  Key: identifier
  [Path: /identifier]
Member: password
  [Path with value: /password:1234]
  [Member with value: password:1234]
  String value: 1234
  Key: password
  [Path: /password]
```

Рисунок 5.7 – Содержание тела запроса

```
POST /api/authorization/login HTTP/1.1 , JSON (application/json)
HTTP/1.1 202 Accepted , JSON (application/json)
GET /setcookie?sessionToken=95a6c1a65d36ee962eb48a18dd3e9674 HTTP/1.1
HTTP/1.1 302 Found (text/html)
GET /main HTTP/1.1
```

Рисунок 5.7 – Следующие запросы при валидных данных

В случае, когда пользователь вводит валидные данные, в ответ он получает сообщение с выделенным ему токеном. После чего, он перенаправляется на специально выделенный адрес, который нужен для установки токена в куки. После чего пользователь перенаправляется на основную страницу.

После успешного входа в систему навигационная панель автоматически перестраивается. Гостевые элементы скрываются, и пользователю становятся доступны основная ссылка для перехода в игровое лобби, персональный профиль, а также кнопка для безопасного завершения сеанса и выхода из учетной записи.

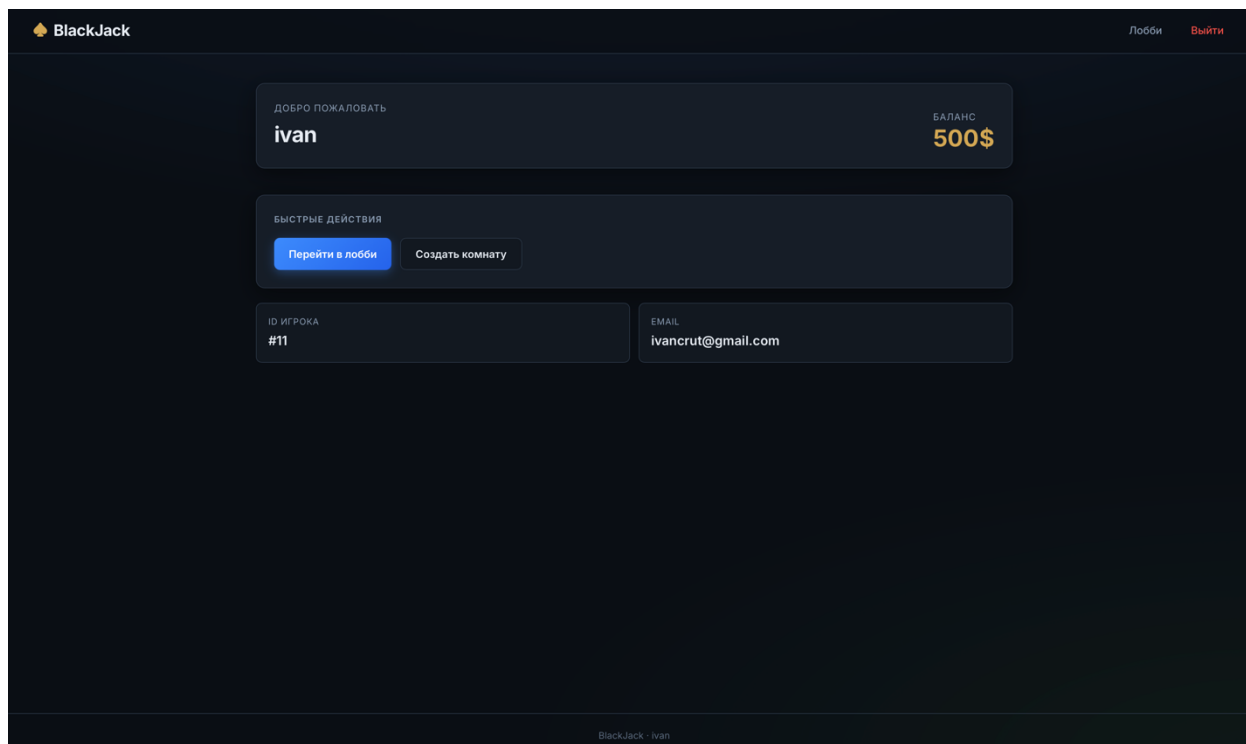


Рисунок 5.8 – Интерфейс главного меню для авторизованного пользователя

Для защиты от случайных действий в систему внедрен механизм обязательного подтверждения. При попытке завершить сеанс через главное меню браузер генерирует диалоговое окно с запросом подтверждения выхода из аккаунта.

Вы уверены, что хотите выйти из аккаунта?

Отменить ОК

Рисунок 5.9 – Диалоговое окно подтверждения выхода из аккаунта

Переход в раздел лобби открывает перед пользователем интерфейс просмотра и поиска доступных игровых столов. На экране отображается список комнат с указанием их названия, описания, текущего количества игроков и количества используемых колод.

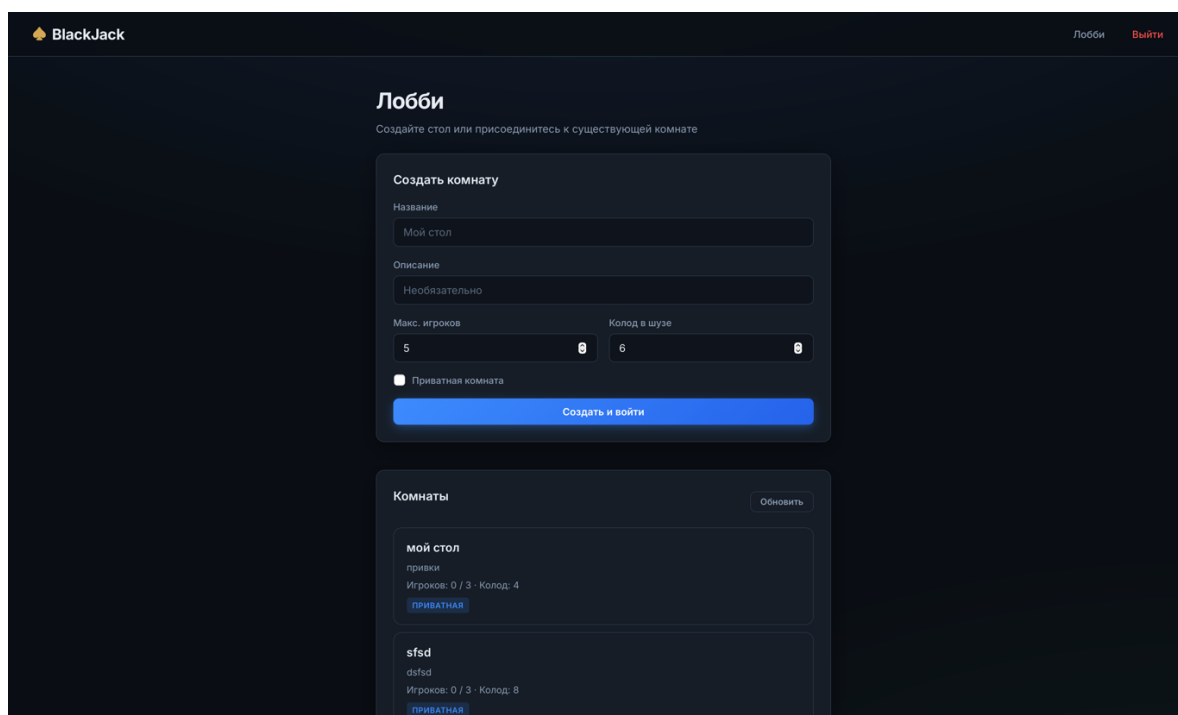


Рисунок 5.10 – Интерфейс игрового лобби со списком доступных комнат

```
GET /lobby/rooms HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.11 – Запрос на получение меню комнат

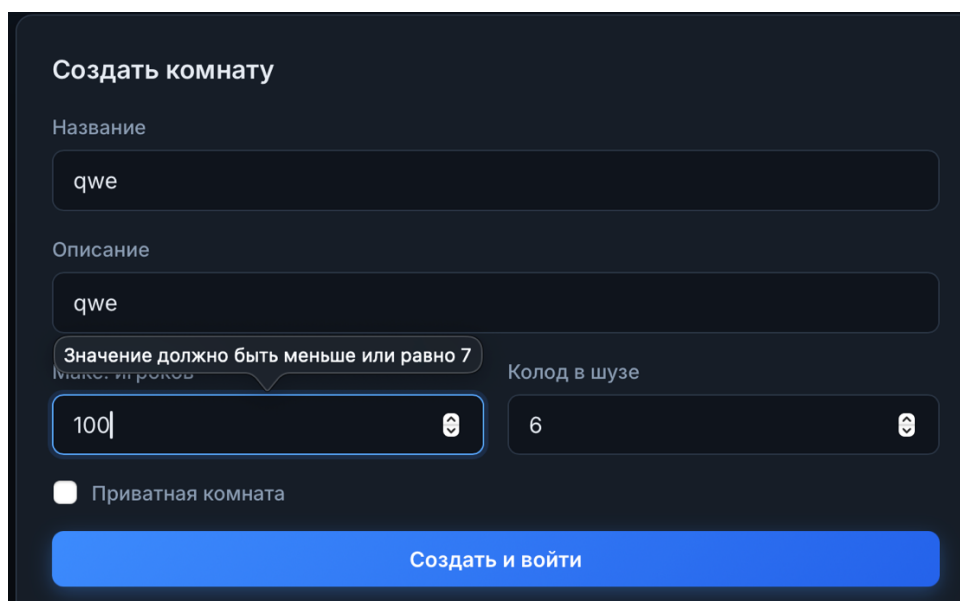
После загрузки всех скриптов страницы, автоматически отправляется запрос на получение всех активных комнат.

```
GET /api/lobby/rooms HTTP/1.1
HTTP/1.1 200 OK , JSON (application/json)
```

Рисунок 5.12 – Дополнительный запрос на получение всех активных комнат

В интерфейсе лобби также присутствует панель конфигурации нового стола. При указании недопустимых параметров в форме создания комнаты

выводится текстовая плашка с описанием ошибки, информируя пользователя о необходимости скорректировать лимиты или название.



The screenshot shows a dark-themed web form titled "Создать комнату" (Create Room). It contains two text input fields: "Название" (Name) with the value "qwe" and "Описание" (Description) with the value "qwe". Below the description field is a tooltip that reads "Значение должно быть меньше или равно 7" (Value must be less than or equal to 7). There are two numeric input fields: "Макс. игроков" (Max. players) with the value "100" and "Колод в шузе" (Deck in shuff) with the value "6". Both numeric fields have a small circular icon with a minus sign to their right. At the bottom left, there is a checkbox labeled "Приватная комната" (Private room). A large blue button at the bottom is labeled "Создать и войти" (Create and login).

Рисунок 5.13 – Индикация ошибки при создании новой игровой комнаты

Вход в существующие приватные комнаты защищен паролем. При попытке подключения к такому столу система вызывает всплывающее диалоговое окно для ввода секретной комбинации символов, предотвращая несанкционированный доступ посторонних участников.

```
POST /api/lobby/rooms HTTP/1.1 , JSON (application/json)
HTTP/1.1 201 Created , JSON (application/json)
```

Рисунок 5.14 – запрос на создание комнаты

В теле запроса на создание комнаты находятся все данные, введенные пользователем для создания конкретной комнаты. В ответ на что пользователь получает ответ, содержащий айди созданной комнаты. После чего, пользователь перенаправляется на её страницу.

```
GET /lobby/room/d29d9807-ef30-47c8-bb53-01c92cb3315a HTTP/1.1
HTTP/1.1 200 OK (text/html)
```

Рисунок 5.15 – запрос на получение страницы созданной комнаты

После успешного получения страницы комнаты, устанавливается WebSocket соединение с применением socket.io.

```
GET /socket.io/?EI0=4&transport=websocket HTTP/1.1
HTTP/1.1 101 Switching Protocols
```

Рисунок 5.16 – запрос на создание комнаты

Введите пароль комнаты:

Отменить OK

Рисунок 5.17 – Всплывающее окно для ввода пароля от приватной комнаты

Интерфейс самой игровой комнаты разделен на зону дилера, где отображаются карты и текущий счет, индивидуальные слоты игроков с их балансом и активными руками, а также панель текстового чата для общения участников.

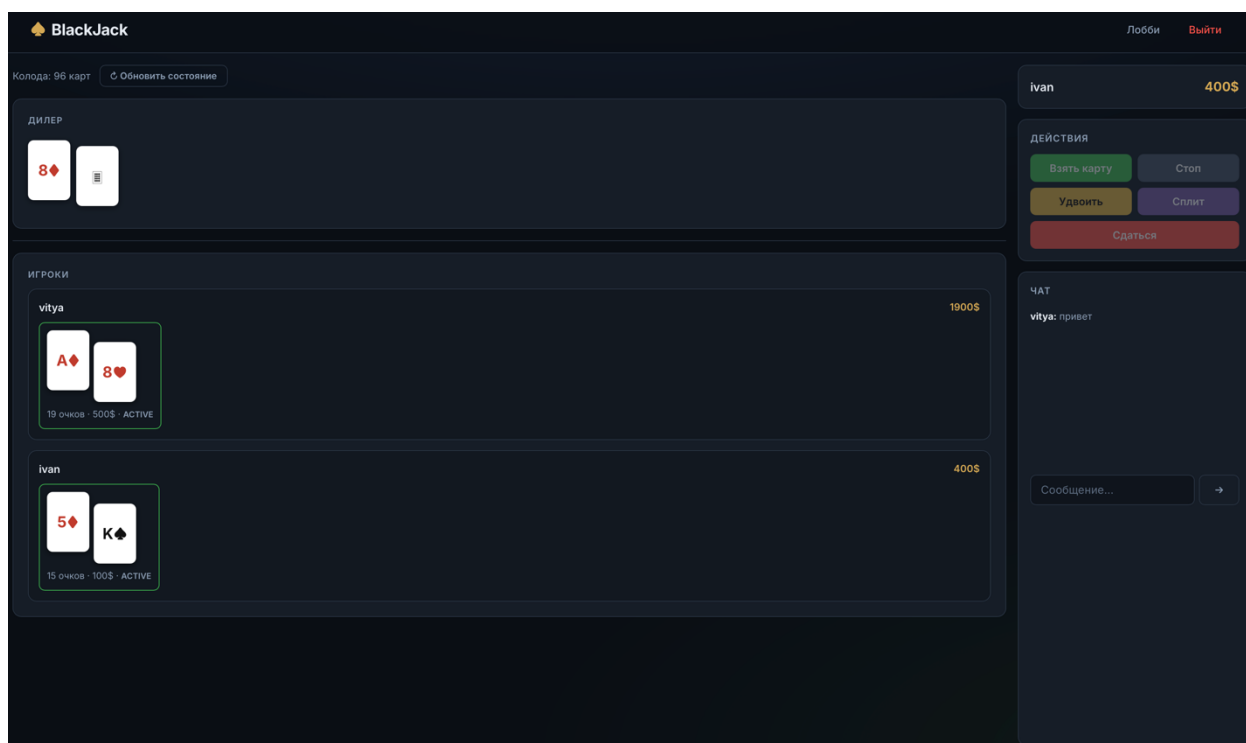


Рисунок 5.18 – Общий интерфейс активной игровой комнаты

Во время фазы приема ставок пользователю становится доступна специальная панель взаимодействия. Игрок может использовать

интерактивные фишки для быстрого ввода суммы или вписать значение вручную в соответствующее поле.

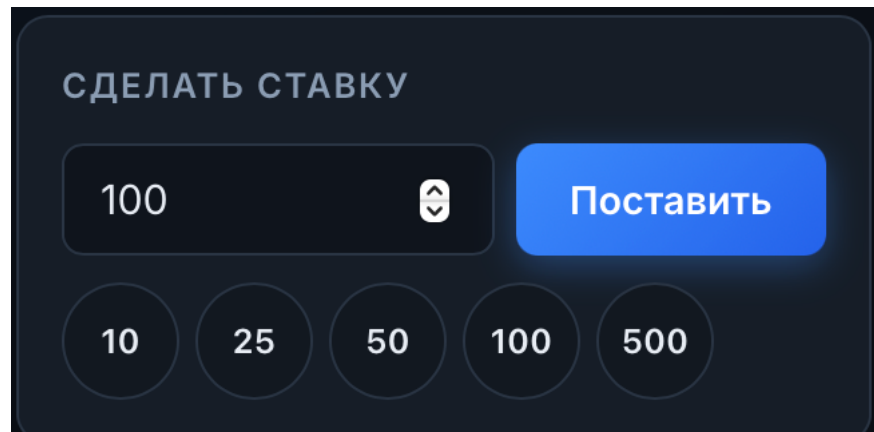


Рисунок 5.19 – Панель интерфейса для выбора и подтверждения ставки

Система строго контролирует процесс совершения ставок. Если пользователь указывает нулевую или отрицательную сумму, на экран высвечивается красная плашка о невозможности совершить ставку.

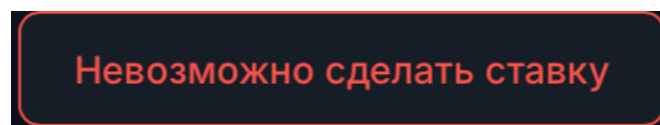


Рисунок 5.20 – Предупреждающее уведомление о некорректной сумме ставки

Теперь, после подключения к вебсокету, внутри комнаты клиент и сервер начинают обмениваться сообщениями на установленном соединении. Выглядит это следующим образом:

```
WebSocket Text [FIN] [MASKED]
```

Рисунок 5.21 – WebSocket сообщение

Содержание таких сообщений всегда похожее, а именно объект с названием совершаемого события и само содержание в формате JSON.

```
42["PLACE_BET",{"roomId":"2d52d5b2-2d2b-469f-ad44-78891dc086e4","amount":50}]
```

Рисунок 5.22 – Содержание сообщений

Для обеспечения непрерывной обратной связи в процессе активной партии реализован механизм динамических всплывающих уведомлений.

Информационные плашки временно появляются в боковой части экрана и автоматически исчезают, не перекрывая обзор виртуального стола.

Синие плашки предназначены для информирования о штатных системных событиях. Они появляются при подключении нового участника к столу, перетасовке колоды или объявлении итогового счета дилера.

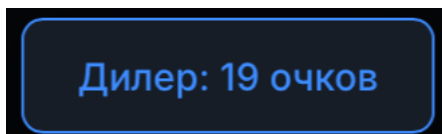


Рисунок 5.23 – Информационное уведомление о внутриигровом событии

Внутри игровой комнаты также активно применяются диалоговые окна для подтверждения критических решений. Если игрок выбирает стратегическое действие досрочной сдачи во время раздачи, система дополнительно информирует его о неминуемой потере половины от сделанной ставки, запрашивая финальное подтверждение.

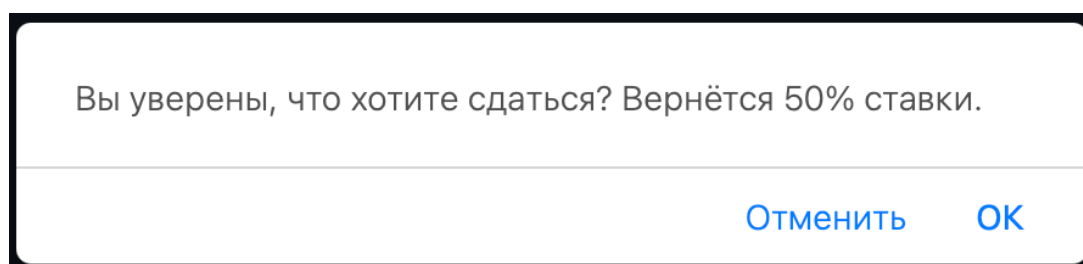


Рисунок 5.24 – Модальное окно подтверждения досрочной сдачи

Аналогичный механизм предупреждения срабатывает при попытке пользователя покинуть активную игровую комнату с помощью кнопки возврата или навигационного меню в заголовке страницы. Это защищает игрока от случайной потери текущего прогресса в раунде.



Рисунок 5.25 – Модальное окно подтверждения выхода из игровой комнаты

По завершении всех ходов в текущем раунде система поверх стола генерирует детализированное модальное окно с финансовыми результатами. В нем построчно отображаются исходы для каждого участника, суммы изменений баланса и общие итоги партии. Данное окно демонстрируется

пользователям ровно шесть секунд и автоматически скрывается перед инициализацией новой фазы приема ставок.

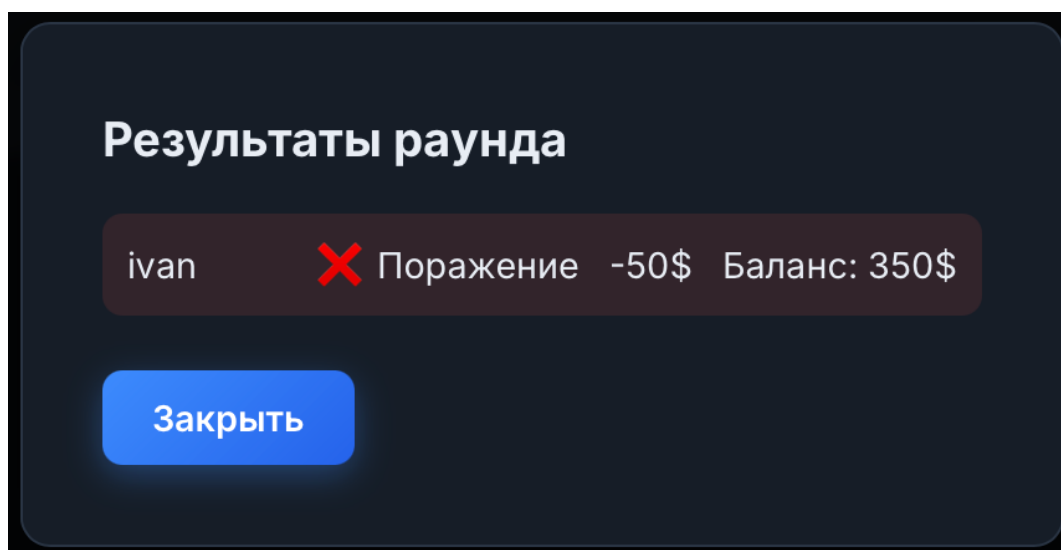


Рисунок 5.26 – Детализированное окно с финансовыми результатами раунда

```
[{"ROUND_RESULT", {"results": [{"userId": 1, "name": "vitya", "result": "lose", "payout": 0, "netProfit": -50}]}
```

Рисунок 5.27 – Содержание сообщения с результатом раунда

ЗАКЛЮЧЕНИЕ

В процессе выполнения данного курсового проекта была успешно спроектирована, реализована и протестирована масштабируемая многопользовательская распределённая программная система реального времени «*blackjack*». Актуальность проведенной работы обусловлена растущим спросом на высоконагруженные интерактивные веб-приложения, требующие мгновенного отклика, отказоустойчивости и надежной синхронизации состояний между множеством независимых клиентов.

В ходе аналитического этапа был проведён детальный анализ предметной области и выявлены ключевые проблемы *stateful*-серверов. Для обхода ограничений классического протокола *http* была выбрана событийно-ориентированная асинхронная парадигма на базе протокола *websocket*. Использование современного технологического стека, включающего среду выполнения *node.js*, фреймворк *express* и библиотеку *socket.io* поверх строго типизированного языка *typescript*, позволило создать надёжное серверное ядро, способное эффективно обрабатывать высококонкурентные потоки и обслуживать запросы с минимальной задержкой.

В рамках проектирования разработана детальная структурная схема слоистой архитектуры и спроектирован комплексный протокол сокет-событий. Он обеспечивает полнодуплексный обмен дельта-обновлениями состояний, существенно минимизируя сетевой трафик. Особое внимание уделено разделению ответственности баз данных: интеграция субд *postgresql* гарантировала долгосрочную персистентность профилей и безопасное изменение баланса, тогда как in-memory хранилище *redis* обеспечило высокопроизводительное кэширование динамических игровых сессий.

В систему внедрена надежная аутентификация на базе *http-only cookie*, защищающая от *xss*-атак, а также криптографическое хэширование паролей. Клиентский интерфейс построен по принципу «тонкого клиента» и управляется потоком серверных событий. Вся критическая игровая логика и математика карточных раздач изолированы на сервере, что фундаментально исключает несанкционированную модификацию данных или использование уязвимостей со стороны клиента.

Комплексное функциональное и нагрузочное тестирование подтвердило абсолютную корректность работы игрового движка и стабильность сессий при высоких нагрузках. Таким образом, поставленные задачи были полностью решены, а цель по созданию безопасной и высокотехнологичной игровой платформы *blackjack* успешно достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Node.js v22 API Reference [Электронный ресурс]. – Режим доступа: <https://nodejs.org/docs/latest-v22.x/api/>.
- [2] Express 5.x API Reference [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/api.html>.
- [3] Express.js. Routing Guide [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/guide/routing.html>.
- [4] The TypeScript Handbook [Электронный ресурс] / Microsoft Corporation. – Режим доступа: <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [5] Socket.IO v4 Documentation [Электронный ресурс]. – Режим доступа: <https://socket.io/docs/v4/>.
- [6] The Socket.IO Protocol Specification [Электронный ресурс]. – Режим доступа: <https://socket.io/docs/v4/socket-io-protocol/>.
- [7] PostgreSQL 17 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/index.html>.
- [8] Redis Documentation: Develop with Redis [Электронный ресурс]. – Режим доступа: <https://redis.io/docs/latest/develop/>.
- [9] The WebSocket Protocol: RFC 6455 [Электронный ресурс] / I. Fette, A. Melnikov. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc6455>.
- [10] Password Storage Cheat Sheet [Электронный ресурс] / OWASP Foundation. – Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.
- [11] Handlebars Language Guide [Электронный ресурс]. – Режим доступа: <https://handlebarsjs.com/guide/>.
- [12] Fisher–Yates shuffle [Электронный ресурс]. – Режим доступа: https://dev.to/tanvir_azad/fisher-yates-shuffle-the-right-way-to-randomize-an-array-4d2p.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода с комментариями

1. ВНУТРЕННИЕ МОДЕЛИ И ФУНКЦИИ, НЕ ОТНОСЯЩИЕСЯ К СЕТИ

=====

src/express/models/user.dto.ts

```
interface User { name: string; email: string; userId: number;
  balance: number; hpassword: string; }
```

-- Полная модель пользователя.

src/express/models/userSession.dto.ts

```
interface UserSession { name: string; email: string;
  userId: number; balance: number; }
```

-- Данные пользователя, хранящиеся в сессии.

src/express/models/room.meta.dto.ts

```
interface RoomMeta { roomId: string; name: string;
  description: string; maxPlayersCount: number;
  currentPlayersCount: number; isPrivate: boolean;
  password: string; deckCount: number; }
```

-- Метаданные игровой комнаты.

src/express/models/room.dto.ts

```
enum room_status { PLAYING = "PLAYING", BETTING = "BETTING" }
```

-- Состояния комнаты.

```
interface Room { roomId: string; dealer: string[];
  status: room_status; currentPlayer: number | null;
  deck: string[]; reshufflePending: boolean;
  playerOrder: number[]; }
```

-- Игровое состояние комнаты.

src/express/models/hand.dto.ts

```
enum hand_status { ACTIVE, STOOD, DOUBLE, SURRENDER,
  BUST, BLACKJACK }
```

-- Состояния руки игрока.

```
interface Hand { bet: number; cards: string[];
  status: hand_status; }
```

-- Данные одной руки игрока.

src/game/BlackjackEngine.ts

```
export function cardValue(card: string): number;
```

-- Возвращает стоимость карты.

```
export function cardRank(card: string): string;
```

-- Возвращает ранг карты.

```
export function countHand(cards: string[]): number;
```

-- Подсчитывает сумму карт с учетом тузов.

```

export function generateDeck(deckCount: number): string[];
-- Генерирует одну или несколько колод.

export function shuffleDeck(deck: string[]): void;
-- Перемешивает колоду.

export function shouldReshuffle(deck: string[]): boolean;
-- Проверяет необходимость перетасовки.

export function canSplit(hand: Hand): boolean;
-- Проверяет возможность разделения пары.

export function canDouble(hand: Hand): boolean;
-- Проверяет возможность удвоения ставки.

export function isNaturalBlackjack(cards: string[]): boolean;
-- Проверяет натуральный Blackjack.

buildRoomState = async (roomId: string, revealDealer = false)
    => Promise<RoomState>;
-- Формирует состояние комнаты для клиентов.

placeBet = async (roomId: string, userId: number, amount:
number)
    => Promise<any | null>;
-- Принимает ставку и списывает баланс.

startGame = async (roomId: string) => Promise<RoomState>;
-- Начинает игровой раунд.

playerHit = async (roomId, userId) => Promise<any | null>;
-- Игрок берет карту.

playerStand = async (roomId, userId) => Promise<any | null>;
-- Игрок завершает руку.

playerDouble = async (roomId, userId) => Promise<any | null>;
-- Игрок удваивает ставку.

playerSplit = async (roomId, userId) => Promise<any | null>;
-- Игрок разделяет пару.

playerSurrender = async (roomId, userId) => Promise<any | null>;
-- Игрок сдается.

nextTurn = async (roomId, currentUserId) => Promise<any>;
-- Передает ход следующему игроку или дилеру.

playDealer = async (roomId: string)
    => Promise<{ cards: string[]; score: number }>;
-- Выполняет ход дилера.

settleRound = async (roomId: string) => Promise<any>;

```

```

-- Подводит итоги раунда.

resetRound = async (roomId: string) => Promise<void>;
-- Сбрасывает раунд.

forfeitPlayer = async (roomId, userId) => Promise<void>;
-- Завершает руки игрока при выходе.

2. СЕТЕВАЯ ЧАСТЬ: ЗАПУСК СЕРВЕРА
=====
src/index.ts
const bootstrap = async () => {
  try {
    await redisClient.connect();
  }
  catch (err) {
    console.error(`failed to connect to redis on startup:
${err}`);
    process.exit(1);
  }

  try {
    const removedRoomsCount =
      await roomRedisRepository.deleteAllRooms();
    if (removedRoomsCount > 0) {
      console.log(`deleted ${removedRoomsCount} rooms from
Redis`);
    }
  }
  catch (err) {
    console.error(`failed to clear redis rooms: ${err}`);
  }

  server.listen(process.env.PORT, () => {
    console.log(`server starts listening on port
${process.env.PORT}`);
  });
};
-- Запускает Redis, очищает временные комнаты и включает HTTP-сервер.

src/server/server.ts
const server = http.createServer(app);
-- Создает HTTP-сервер на основе Express-приложения.

const socketio = new Server(server, { cors: {
  origin: ["http://127.0.0.1:8888", "*"],
  methods: ["GET", "POST"]
}});
-- Создает Socket.IO-сервер и задает CORS-настройки.

socketio.on("connection", async (socket: Socket) => {

```

```

    const cookies = cookie.parse(socket.handshake.headers.cookie
|| "");
    const sessionToken = cookies["sessionToken"];

    if (!sessionToken) {
        return;
    }
});
-- При подключении socket-клиента считывает cookie и проверяет
token.

```

3. СЕТЕВАЯ ЧАСТЬ: EXPRESS И HTTP-МАРШРУТЫ

```

=====
src/server/express.ts
const app = express();
-- Создает экземпляр Express-приложения.

app.use(express.static(path.join(__dirname, "../..public")));
-- Раздает статические файлы клиенту.

app.engine('hbs', hbs.express4({
    partialsDir: path.join(__dirname, "../..views/partials"),
    layoutsDir: path.join(__dirname, "../..views/layouts")
}));
-- Подключает шаблонизатор Handlebars.

app.set('view engine', 'hbs');
app.set('views', path.join(__dirname, "../..views"));
-- Указывает движок шаблонов и папку представлений.

HandlebarsHelpers(hbs);
-- Регистрирует пользовательские helpers для шаблонов.

app.get('/setcookie', setCookie);
-- Сетевой маршрут для установки cookie с sessionToken.

app.use(cp());
-- Подключает cookie-parser для чтения cookie из HTTP-запросов.

app.use(AuthorizationMiddleware.checkUserByCookie);
-- Для каждого запроса проверяет авторизацию по cookie.

app.use(refreshCookieMiddleware.refreshSession);
-- Продлевает срок действия активной сессии.

app.use("/authorization", authorizationViewRouter);
-- Подключает страницы входа и регистрации.

app.use("/lobby", checkForAuthenticationMiddleware.checkUser,
    roomViewRouter);
-- Подключает защищенные страницы лобби.

app.use("/main", MainViewRouter);

```

```

-- Подключает главную страницу.

app.use(express.json());
-- Включает обработку JSON-тела запросов для API.

app.use("/api/lobby", checkForAuthAPIMiddleware.checkUser,
roomsApiRouter);
-- Подключает защищенное API комнат.

app.use("/api/authorization", authorizationApiRouter);
-- Подключает API авторизации.

app.use("/", (req, res) => {
    res.redirect("/main");
});
-- Все неизвестные маршруты перенаправляет на главную страницу.

```

4. СЕТЕВАЯ ЧАСТЬ: COOKIE И MIDDLEWARE

=====

```

src/express/support/setCookie.support.ts
const setCookie = async (req: Request, res: Response) => {
    const sessionToken = req.query["sessionToken"];

    res.cookie("sessionToken", sessionToken, {
        maxAge: EXPIRATION_TIME.cookie,
        path: "/",
        httpOnly: true
    });

    res.redirect("/main");
};
-- Получает sessionToken из URL, записывает его в cookie и
делает
    переход на главную страницу.

```

```

src/express/middlewares/authentication.middleware.ts
checkUserByCookie = async (req, res, next) => {
    const sessionToken = req.cookies["sessionToken"];

    if (!sessionToken) {
        req.userSession = null;
        return next();
    }

    const userSession =
        await
UserSessionService.getUserSessionByToken(sessionToken);

    if (!userSession) {
        req.userSession = null;
        return next();
    }
}

```

```

    req.userSession = userSession;
    return next();
};
-- Проверяет cookie sessionToken и записывает данные
пользователя
    в req.userSession для следующих обработчиков.

src/express/middlewares/refreshCookie.middleware.ts
refreshSession = async (req, res, next) => {
    const sessionToken = req.cookies["sessionToken"];

    if (!sessionToken) {
        return next();
    }

    const userSession =
        await
UserSessionService.refreshSessionByToken(sessionToken);

    if (!userSession) {
        res.clearCookie("sessionToken", { path: "/" });
        return next();
    }

    res.cookie("sessionToken", sessionToken, {
        maxAge: EXPIRATION_TIME.cookie,
        path: "/",
        httpOnly: true
    });

    return next();
};
-- Продлевает активную сессию, а при ошибке очищает cookie.

```

```

src/express/middlewares/checkForAuthAPI.middleware.ts
checkUser = async (req, res, next) => {
    if (!req.userSession) {
        res.status(401).json({ message: "you need to login
first!" });
        return;
    }
}

```

```

    next();
};
-- Блокирует API-запросы от неавторизованных пользователей.

```

```

src/express/middlewares/checkForAuthentication.middleware.ts
checkUser = async (req, res, next) => {
    if (!req.userSession) {
        res.redirect("/authorization/register");
        return;
    }
}

```

```

    next();
};
-- Если пользователь не авторизован, отправляет его на
регистрацию.

src/express/middlewares/checkForUserInRoom.middleware.ts
checkUser = async (req, res, next) => {
    const roomId = req.params["roomId"] as string;

    if (!roomId || !req.userSession) {
        res.redirect("/main");
        return;
    }

    const users = await roomService.getAllUsersFromRoom(roomId);
    const includes = users.some(
        el => Number(el) === req.userSession?.userId
    );

    if (!includes) {
        res.redirect("/main");
        return;
    }

    next();
};
-- Проверяет, что авторизованный пользователь состоит в комнате.

5. СЕТЕВАЯ ЧАСТЬ: API АВТОРИЗАЦИИ
=====
src/express/api/controllers/authorization.api.controller.ts
private kickFromRoom = async (userId: number, reason: string) =>
{
    try {
        const { socketio } = await
import("../.../server/server.js");
        const sockets = await socketio.fetchSockets();

        for (const s of sockets) {
            if ((s.data as any)?.userSession?.userId === userId)
            {
                s.emit("FORCE_DISCONNECT", { reason });
                s.disconnect();
            }
        }
    }
    catch {
        /* socket может быть не инициализирован */
    }

    await roomService.removeUserFromCurrentRoom(userId);
};
-- Отключает старые socket-соединения пользователя и удаляет его

```

из текущей комнаты.

```
login = async (req, res, next) => {
  if (!req.body) {
    res.json({ message: "Empty req.body" });
    return;
  }

  const { identifier, password } = req.body;

  if (!identifier || !password) {
    res.status(400).json({
      message: "Пустые значения для identifier или
password"
    });
    return;
  }

  const userSession =
    await userService.authenticate(identifier, password);

  if (!userSession) {
    res.status(404).json({ message: "Неверный логин или
пароль" });
    return;
  }

  await this.kickFromRoom(userSession.userId);

  const token = await
userService.setSession(userSession);

  if (!token) {
    res.status(500).json({ message: "Ошибка при создании
сессии" });
    return;
  }

  res.status(202).json({ message: "all good", sessionToken:
token });
};
-- Обрабатывает вход: проверяет данные, создает сессию и отдает
token.

register = async (req, res, next) => {
  if (!req.body) {
    res.json({ message: "Empty req.body" });
    return;
  }

  const { name, email, password } = req.body;

  if (!name || !email || !password) {
```

```

        res.status(400).json({
            message: "Пустые значения для name, email или
password"
        });
        return;
    }

    const userSession = await userService.addUser(name, email,
password);

    if (!userSession) {
        res.status(400).json({
            message: "Пользователь с такими данными уже
существует"
        });
        return;
    }

    const token = await
userSessionService.setSession(userSession);

    if (!token) {
        res.status(500).json({ message: "Ошибка при создании
сессии" });
        return;
    }

    res.status(202).json({ message: "all good", sessionToken:
token });
};
-- Обработывает регистрацию: создает пользователя, сессию и
token.

logout = async (req, res, next) => {
    const sessionToken = req.cookies["sessionToken"];

    if (!sessionToken) {
        res.status(400).json({ message: "Нет активной сессии"
});
        return;
    }

    if (req.userSession?.userId) {
        await this.kickFromRoom(req.userSession.userId,
            "Вы вышли из аккаунта");
    }

    const result =
        await
userSessionService.removeSessionByToken(sessionToken);

    if (!result) {
        res.status(400).json({ message: "Сессия не найдена" });
    }
}

```

```

        return;
    }

    res.status(200).json({ message: "all good" });
};
-- Выполняет выход: отключает игрока и удаляет сессию.

src/express/api/routers/authorization.api.router.ts
initialRoutes() {
    this.router.post("/login",
AuthorizationApiController.login);
    this.router.post("/register",
AuthorizationApiController.register);
    this.router.post("/logout",
AuthorizationApiController.logout);
}
-- Связывает HTTP-адреса авторизации с методами контроллера.

```

6. СЕТЕВАЯ ЧАСТЬ: API КОМНАТ

```

=====
src/express/api/controllers/room.api.controller.ts
createRoom = async (req, res, next) => {
    if (!req.userSession) {
        res.status(401).json({ message: "not logged" });
        return;
    }

    const body = req.body ?? {};
    const result = await roomService.createRoomForUser(
        req.userSession.userId,
        {
            name: body.name,
            description: body.description,
            maxPlayersCount: body.maxPlayersCount,
            isPrivate: body.isPrivate,
            password: body.password,
            deckCount: body.deckCount,
        }
    );

    if ("error" in result) {
        res.status(400).json({ message: result.error });
        return;
    }

    res.status(201).json({ message: "all good", roomId:
result.roomId });
};
-- Создает комнату для авторизованного пользователя.

getAllRooms = async (req, res, next) => {
    const data = await roomService.getAllRoomsMeta();

```

```

    if (!data) {
        res.json([]);
    }
    else {
        res.json(data);
    }
};
-- Возвращает клиенту список активных комнат.

enterRoom = async (req, res, next) => {
    if (!req.userSession) {
        res.json({ message: "not loggedin" });
        return;
    }

    if (!req.body) {
        res.json({ message: "emptyBody" });
        return;
    }

    const roomId = req.params["roomId"] as string;
    const password = req.body?.password ?? null;

    if (!roomId) {
        res.json({ message: "invalid input" });
        return;
    }

    const result = await roomService.addUserToRoom(
        roomId, req.userSession.userId, password
    );

    if (!result) {
        res.json({ message: "error when attempt to join!" });
        return;
    }

    res.json({ message: "all good" });
};
-- Добавляет пользователя в комнату по roomId.

leaveRoom = async (req, res, next) => {
    if (!req.userSession) {
        res.json({ message: "not loggedin" });
        return;
    }

    const userId = req.userSession.userId;
    const roomId = await
roomService.removeUserFromCurrentRoom(userId);

    if (!roomId) {
        res.json({ message: "you out from any room!" });
    }
};

```

```

        return;
    }

    res.json({ message: "all good", roomId: roomId });
};
-- Удаляет пользователя из текущей комнаты.

deleteRoom = async (req, res, next) => {
    if (!req.userSession) {
        res.status(401).json({ message: "not logged" });
        return;
    }

    const roomId = req.params["roomId"] as string;

    if (!roomId) {
        res.status(400).json({ message: "invalid input" });
        return;
    }

    const deleted = await roomService.deleteEmptyRoom(roomId);

    if (!deleted) {
        res.status(409).json({
            message: "room is not empty or does not exist"
        });
        return;
    }

    res.json({ message: "all good" });
};
-- Удаляет комнату, если она пуста.

src/express/api/routers/rooms.api.router.ts
initialRoutes() {
    this.router.post("/rooms", RoomsApiController.createRoom);
    this.router.put("/room/leave",
RoomsApiController.leaveRoom);
    this.router.get("/rooms", RoomsApiController.getAllRooms);
    this.router.put("/room/:roomId",
RoomsApiController.enterRoom);
    this.router.delete("/room/:roomId",
RoomsApiController.deleteRoom);
}
-- Регистрирует API-маршруты для работы с комнатами.

```

7. СЕТЕВАЯ ЧАСТЬ: ОТОБРАЖЕНИЕ HTML-СТРАНИЦ

```

=====
src/express/views/controllers/authorization.view.controller.ts
renderLoginPage = async (req, res, next) => {
    res.render("authorization", {
        layout: "layout",
        scripts: "loginScripts",
    });
};

```

```

        partial: "loginView",
        user: req.userSession
    });
};
-- Отдает HTML-страницу входа.

renderRegistrationPage = async (req, res, next) => {
    res.render("authorization", {
        layout: "layout",
        scripts: "registerScripts",
        partial: "registerView",
        user: req.userSession
    });
};
-- Отдает HTML-страницу регистрации.

src/express/views/controllers/main.view.controller.ts
renderPage = async (req, res, next) => {
    res.render("main", {
        layout: "layout",
        user: req.userSession,
    }, (err, html) => {
        console.log("error:", err);
        res.send(html);
    });
};
-- Отдает главную страницу приложения.

src/express/views/controllers/room.view.controller.ts
RenderRoomsPage = async (req, res, next) => {
    const data = await roomService.getAllRoomsMeta() || [];
    res.render("rooms", {
        layout: "layout",
        rooms: data,
        user: req.userSession
    });
};
-- Отдает страницу со списком комнат.

RenderRoomPage = async (req, res, next) => {
    const roomId = req.params["roomId"];
    res.render("room", {
        layout: "roomLayout",
        user: req.userSession,
        roomId,
        userId: req.userSession?.userId,
    });
};
-- Отдает страницу конкретной игровой комнаты.

```

8. СЕТЕВАЯ ЧАСТЬ: SOCKET.IO MIDDLEWARE

```

=====
src/socket.io/socket.middleware.ts

```

```

export const authenticateSocket = async (socket, next) => {
  const cookies = cookie.parse(socket.handshake.headers.cookie
  || "");
  const sessionToken = cookies["sessionToken"];

  if (!sessionToken) {
    socket.emit("SESSION_INVALID");
    return next(new Error("Not authenticated"));
  }

  const session =
    await
userSessionService.getUserSessionByToken(sessionToken);

  if (!session) {
    socket.emit("SESSION_INVALID");
    return next(new Error("Session invalid or expired"));
  }

  socket.data.userSession = session;
  socket.data.sessionToken = sessionToken;
  socket.data.currentRoomId = null;

  next();
};

```

-- Авторизует socket-подключение по sessionToken из cookie.

```

export const guardSession = async (socket, next) => {
  const token = socket.data.sessionToken;

  if (!token) {
    socket.emit("SESSION_INVALID");
    socket.disconnect();
    return;
  }

  const session = await
userSessionService.getUserSessionByToken(token);

  if (!session) {
    socket.emit("SESSION_INVALID");
    socket.disconnect();
    return;
  }

  socket.data.userSession = session;
  next();
};

```

-- Проверяет актуальность сессии перед каждым socket-событием.

9. СЕТЕВАЯ ЧАСТЬ: SOCKET.IO СОБЫТИЯ

```

=====
src/socket.io/socket.io.ts

```

```

io.use(authenticateSocket as any);
-- Подключает авторизацию ко всем socket-подключениям.

async function getPlayerBalance(userId: number): Promise<number>
{
    const sessionToken = await
userSessionRepo.getSessionByUserId(userId);
    if (!sessionToken) return 0;

    const session = await
userSessionRepo.getUserSessionByToken(sessionToken);
    return session?.balance ?? 0;
}
-- Получает актуальный баланс игрока.

async function handleNextTurn(roomId: string, userId: number) {
    const turn = await engine.nextTurn(roomId, userId);

    if ("nextUserId" in turn) {
        io.to(roomId).emit("TURN_CHANGED", {
            userId: turn.nextUserId,
            currentHandIndex: turn.currentHandIndex
        });
        return;
    }

    if ("dealerPlaying" in turn) {
        const dealerResult = await engine.playDealer(roomId);
        io.to(roomId).emit("DEALER_PLAY", dealerResult);

        const settle = await engine.settleRound(roomId);

        io.to(roomId).emit("ROUND_RESULT", {
            results: settle.results,
            dealer: {
                cards: settle.dealerCards,
                score: settle.dealerScore
            },
        });

        for (const r of settle.results) {
            io.to(roomId).emit("PLAYER_BALANCE", {
                userId: r.userId,
                balance: r.newBalance
            });
        }

        if (settle.reshuffled) {
            const repo = await import(
"../express/repositories/room.redis.repository.js"
            );
            const g = await repo.default.getRoomGame(roomId);

```

```

        io.to(roomId).emit("DECK_SHUFFLED", {
            remainingCards: g?.deck.length ?? 0
        });
    }

    await engine.resetRound(roomId);

    setTimeout(async () => {
        io.to(roomId).emit("BETTING_PHASE", {});
        const state = await engine.buildRoomState(roomId);
        io.to(roomId).emit("ROOM_STATE", state);
    }, 3000);
}
}

-- Передает ход, запускает дилера, подводит итоги и рассылает
   сетевые события всем игрокам комнаты.

async function emitGameStarted(roomId: string): Promise<void> {
    const gameState = await engine.startGame(roomId);
    io.to(roomId).emit("GAME_STARTED", gameState);

    const firstPlayer = gameState.currentPlayerId;

    if (firstPlayer !== null) {
        const fp = gameState.players.find(p => p.userId ===
firstPlayer);
        if (fp?.hands.every(h => h.status !== "ACTIVE")) {
            await handleNextTurn(roomId, firstPlayer);
        }
        return;
    }

    const advance = await engine.isPlayerPhaseComplete(roomId);
    if (advance) {
        await handleNextTurn(roomId, advance.fromUserId);
    }
}

-- Запускает игру и отправляет клиентам событие GAME_STARTED.

async function startGameIfAllBetsPlaced(roomId: string)
    : Promise<boolean> {
    const game = await roomRepo.getRoomGame(roomId);
    if (!game || game.status !== room_status.BETTING) return
false;

    const userIds = await roomRepo.getAllUsersFromRoom(roomId);
    if (userIds.length === 0) return false;

    const allStates = await
roomRepo.getAllPlayersStates(roomId);
    const allBetsPlaced = userIds.every((uid) => {
        const state = allStates.get(Number(uid));

```

```

        return Boolean(state?.hands[0]?.bet &&
state.hands[0].bet > 0);
    });

    if (!allBetsPlaced) return false;

    await emitGameStarted(roomId);
    return true;
}
-- Проверяет ставки всех игроков и запускает игру при
готовности.

io.on("connection", async (socket) => {
    const { userSession } = socket.data;
    if (!userSession) return;

    socket.use(async ([event, ...args], next) => {
        await guardSession(socket as any, next);
    });

    console.log(`[socket] connected:
userId=${userSession.userId}`);

    socket.on("ROOM_JOIN", async ({ roomId }) => {
        const { userSession } = socket.data;
        if (!userSession) return;

        if (disconnectTimers.has(userSession.userId)) {
clearTimeout(disconnectTimers.get(userSession.userId));
            disconnectTimers.delete(userSession.userId);
        }

        const users = await
roomService.getAllUsersFromRoom(roomId);
        if (!users.includes(String(userSession.userId))) {
            socket.emit("ERROR", {
                code: "NOT_IN_ROOM",
                message: "Вы не в этой комнате"
            });
            return;
        }

        if (socket.data.currentRoomId &&
            socket.data.currentRoomId !== roomId) {
            socket.leave(socket.data.currentRoomId);
        }

        await socket.join(roomId);
        socket.data.currentRoomId = roomId;

        const state = await engine.buildRoomState(roomId);
        socket.emit("ROOM_STATE", state);
    });

```

```

        socket.to(roomId).emit("PLAYER_JOINED", {
            userId: userSession.userId,
            name: userSession.name,
        });
    });

    socket.on("ROOM_LEAVE", async () => {
        await handleLeave(socket);
    });

    socket.on("disconnect", async () => {
        await handleLeave(socket, true);

        const { userSession, currentRoomId } = socket.data;
        if (!userSession) return;

        const timer = setTimeout(async () => {
            try {
                if (currentRoomId) {
                    await engine.forfeitPlayer(
                        currentRoomId, userSession.userId
                    );
                    io.to(currentRoomId).emit("PLAYER_LEFT", {
                        userId: userSession.userId
                    });
                }

                await roomService.removeUserFromCurrentRoom(
                    userSession.userId
                );

                if (currentRoomId) {
                    const gameStarted =
                        await
startGameIfAllBetsPlaced(currentRoomId);
                    if (!gameStarted) {
                        const state =
                            await
engine.buildRoomState(currentRoomId);
                        io.to(currentRoomId).emit("ROOM_STATE",
state);
                    }
                }
            } catch (err) {
                console.error("[disconnect timeout error]",
err);
            }

            disconnectTimers.delete(userSession.userId);
        }, 15000);
    });

```

```

        disconnectTimers.set(userSession.userId, timer);
    });

    socket.on("GET_ROOM_STATE", async (payload, ack) => {
        const { userSession } = socket.data;
        if (!userSession) {
            ack?>({ ok: false, message: "Не авторизован" });
            return;
        }

        const roomId = payload?.roomId ??
socket.data.currentRoomId;
        if (!roomId) {
            ack?>({ ok: false, message: "Комната не указана" });
            return;
        }

        try {
            const game = await roomRepo.getRoomGame(roomId);
            if (!game) {
                ack?>({ ok: false, message: "Игра не найдена"
});
                return;
            }

            const revealDealer = game.status !==
room_status.PLAYING;
            const state = await engine.buildRoomState(roomId,
revealDealer);
            socket.emit("ROOM_STATE", state);
            ack?({ ok: true });
        }
        catch (err) {
            socket.emit("ERROR", {
                code: "STATE_FAILED",
                message: "Не удалось обновить состояние"
            });
            ack?({ ok: false, message: "Не удалось обновить
состояние" });
        }
    });

    socket.on("PLACE_BET", async ({ roomId, amount }) => {
        const { userSession } = socket.data;
        if (!userSession) return;

        const result = await engine.placeBet(
            roomId, userSession.userId, amount
        );

        if (!result) {
            socket.emit("ERROR", {
                code: "BET_FAILED",

```

```

        message: "Невозможно сделать ставку"
    });
    return;
}

    socket.emit("BET_CONFIRMED", { balance:
result.newBalance });
    socket.to(roomId).emit("PLAYER_BALANCE", {
        userId: userSession.userId,
        balance: result.newBalance,
    });

    if (result.allBetsPlaced) {
        await emitGameStarted(roomId);
    }
});

socket.on("HIT", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;

    const result = await engine.playerHit(roomId,
userSession.userId);
    if (!result) {
        socket.emit("ERROR", {
            code: "ACTION_FAILED",
            message: "Нельзя взять карту"
        });
        return;
    }

    io.to(roomId).emit("PLAYER_ACTION", {
        userId: userSession.userId,
        action: "HIT",
        hands: result.hands,
        currentHandIndex: result.currentHandIndex,
    });

    const currentHand =
result.hands[result.currentHandIndex];
    if (currentHand?.status === "BUST") {
        await handleNextTurn(roomId, userSession.userId);
    }
});

socket.on("STAND", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;

    const result = await engine.playerStand(roomId,
userSession.userId);
    if (!result) {
        socket.emit("ERROR", {

```

```

        code: "ACTION_FAILED",
        message: "Нельзя остановиться"
    });
    return;
}

io.to(roomId).emit("PLAYER_ACTION", {
    userId: userSession.userId,
    action: "STAND",
    hands: result.hands,
    currentHandIndex: result.currentHandIndex,
});

await handleNextTurn(roomId, userSession.userId);
});

socket.on("DOUBLE", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;

    const result = await engine.playerDouble(roomId,
userSession.userId);
    if (!result) {
        socket.emit("ERROR", {
            code: "ACTION_FAILED",
            message: "Нельзя удвоить"
        });
        return;
    }

    io.to(roomId).emit("PLAYER_ACTION", {
        userId: userSession.userId,
        action: "DOUBLE",
        hands: result.hands,
        currentHandIndex: result.currentHandIndex,
        balance: await getPlayerBalance(userSession.userId),
    });

    await handleNextTurn(roomId, userSession.userId);
});

socket.on("SPLIT", async ({ roomId }) => {
    const { userSession } = socket.data;
    if (!userSession) return;

    const result = await engine.playerSplit(roomId,
userSession.userId);
    if (!result) {
        socket.emit("ERROR", {
            code: "ACTION_FAILED",
            message: "Нельзя сделать сплит"
        });
        return;
    }

```

```

    }

    io.to(roomId).emit("PLAYER_ACTION", {
      userId: userSession.userId,
      action: "SPLIT",
      hands: result.hands,
      currentHandIndex: result.currentHandIndex,
      balance: await getPlayerBalance(userSession.userId),
    });

    io.to(roomId).emit("TURN_CHANGED", {
      userId: userSession.userId,
      currentHandIndex: result.currentHandIndex
    });
  });

socket.on("SURRENDER", async ({ roomId }) => {
  const { userSession } = socket.data;
  if (!userSession) return;

  const result = await engine.playerSurrender(
    roomId, userSession.userId
  );
  if (!result) {
    socket.emit("ERROR", {
      code: "ACTION_FAILED",
      message: "Нельзя сдаться"
    });
    return;
  }

  io.to(roomId).emit("PLAYER_ACTION", {
    userId: userSession.userId,
    action: "SURRENDER",
    hands: result.hands,
    currentHandIndex: result.currentHandIndex,
    balance: await getPlayerBalance(userSession.userId),
  });

  await handleNextTurn(roomId, userSession.userId);
});

socket.on("CHAT_MESSAGE", ({ roomId, text }) => {
  const { userSession } = socket.data;
  if (!userSession || !text.trim()) return;

  io.to(roomId).emit("CHAT_MESSAGE", {
    userId: userSession.userId,
    name: userSession.name,
    text: text.slice(0, 300),
  });
});
});
});

```

```

-- Основной обработчик socket-подключения. Внутри него описаны
  события комнаты, ставок, действий игрока, чата и отключения.

async function handleLeave(socket, isDisconnect = false) {
  const { userSession, currentRoomId } = socket.data;
  if (!userSession || !currentRoomId) return;

  if (!isDisconnect) {
    try {
      await engine.forfeitPlayer(currentRoomId,
userSession.userId);
      const repo = await import(
"../express/repositories/room.redis.repository.js"
      );
      const game = await
repo.default.getRoomGame(currentRoomId);

      if (game?.currentPlayer === userSession.userId) {
        await handleNextTurn(currentRoomId,
userSession.userId);
      }
    }
    catch { }

    socket.to(currentRoomId).emit("PLAYER_LEFT", {
      userId: userSession.userId
    });

    await roomService.removeUserFromRoom(
      currentRoomId, userSession.userId
    );
    socket.leave(currentRoomId);
    socket.data.currentRoomId = null;

    const gameStarted = await
startGameIfAllBetsPlaced(currentRoomId);
    if (!gameStarted) {
      const state = await
engine.buildRoomState(currentRoomId);
      io.to(currentRoomId).emit("ROOM_STATE", state);
    }
  }
}

-- Обрабатывает выход игрока из socket-комнаты и обновляет
состояние.

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы